FUNSCIENTIA · KINGDOMS OF DISCOVERY

Online Saves: Round 2

Three small backend tasks before the jury plays: keep the database awake, add the new statistics, and write the privacy note with a clean-up rule

257F-T76R

A jury starting point. In Task 1, a daily request loads it to keep the database awake.

For	**Djihane**, backend
From	Hicham (game development), with Dounia (project lead)
Date	27 October 2026 · round 2
Please deliver by	**Wednesday 28 October 2026** (the Thirduni deadline is 31 October)
Time needed	about half a day
Comes with	`Funscientia_Round2_Kit.zip`: every file in this guide, ready to use

Contents

1. The three tasks in one page
2. Where to work: your project or the team's
3. Before you start
4. Task 1: Keep the database awake
5. Task 2: The new statistics
6. Task 3: Privacy note and clean-up
7. Deliver your work
8. Troubleshooting and SQL notes
9. Appendix: the full files

1. The three tasks in one page

Your online saves are live in the game since 23 October: every player gets a save code, and the jury has six ready-made starting points. Thank you! Since then the game has grown, and the saves now carry two new fields (see Task 2). Before the jury plays, three small things are left:

Task	Why	Where	Time
1. Keep the database awake	Free Supabase projects are **paused** when nobody uses them for about a week. The jury may play days or weeks after 31 October: if the project is paused, saving and "Continue with my code" fail. A small daily request keeps it active.	cron-job.org, pointing at the **team** project (read-only)	30 min
2. The new statistics	Saves now record each mission's **best stars** and the **furthest place** reached. Four new read-only queries turn them into numbers for our submission: how well players did, how far they got, and how many predicted correctly.	**Your own** project, with sample saves	1 to 2 hours
3. Privacy note and clean-up	We store children's play data. A short, plain privacy note says exactly what is kept, and a clean-up rule deletes saves nobody has used for 12 months.	**Your own** project only	1 to 2 hours

What you deliver

Like last time: one **pull request** on GitHub that changes **only the** `backend/` **folder**:

- `backend/stats.sql`: your file, with the four new queries added at the end;
- `backend/cleanup.sql`: new;
- `backend/PRIVACY.md`: new;
- `backend/sample_saves.sql`: new (test data, for anyone's own test project);
- `backend/README.md`: three short new sections;
- screenshots in `backend/screenshots/` that prove each task works.

> **The one rule that matters.** Never put the **secret** key or the `service_role` key in any file, screenshot, cron job or message. Everything in this guide needs only the **publishable** key (`sb_publishable_…`), which is public by design: it is already inside the game.

2. Where to work: your project or the team's

There are two Supabase projects. Use the right one for each step:

Your own project (from round 1)

- Your test project, with the `saves` table and the three functions.
- **Tasks 2 and 3 are built and tested here**, with the sample saves of this guide.
- You can try anything: nothing here reaches the players.

The team project (`rhttjhqssvzxvdcrkkrv`)

- The real one: the game and the jury use it.
- **Only Task 1 touches it**, and only by **reading** one jury save once a day (no login needed: the cron job uses the public address and the publishable key).
- You never run SQL on it. After our review, Hicham runs your statistics there.

Never run `cleanup.sql` on the team project. It deletes rows. It is tested on your own project only; whether and when it runs on the team project is decided after the review.

No need for the team project's dashboard. You do not need an invitation to the team project for any of these tasks. If you would like to see the real statistics later, ask Hicham: he can run them, or invite you.

3. Before you start

1. **Get the latest code.** In your copy of the repository: `git checkout main`, then `git pull`. Your round-1 files are in `backend/` (merged in pull request #1).
2. **Make a new branch** for this round: `git checkout -b backend-round2`.
3. **Unzip the kit** `Funscientia_Round2_Kit.zip`. It contains `sample_saves.sql`, `stats_round2.sql`, `cleanup.sql` and `PRIVACY_template.md`.
4. **Check your own project still works.** Open it on supabase.com. If it was **paused** (free projects pause when unused), click **Restore project** and wait a few minutes. Open **Table Editor**: the `saves` table must be there.
   If the project is gone, create a new one and run `backend/schema.sql`, `functions.sql` and `security.sql` again, in that order, as in round 1.
5. **Find the publishable key** of the **team** project for Task 1: it is in the repository, in `src/config.ts` (it starts with `sb_publishable_`). Ask Hicham if you cannot find it.

4. Task 1: Keep the database awake

The idea

Supabase's free plan pauses a project after about a week with no activity, and a paused project answers nothing until someone restores it by hand. We set up a free online scheduler, **cron-job.org**, to call the team project's `load_progress` function once a day with a jury code. That counts as activity, and it only **reads**: it never creates or changes a save.

> **Why a jury code, and why it is safe.** `load_progress` is one of the three functions that are open to the public anyway (the game calls it). With a jury code, the answer is a starting point that never changes. Nothing is written, and no player's data is read.

Steps

1. Go to **cron-job.org** and create a free account (an email and a password). Confirm your email.
2. In the dashboard, click **Create cronjob**.
3. **Common** tab:
   - **Title:** `Funscientia keep-alive`
   - **URL:** `https://rhttjhqssvzxvdcrkkrv.supabase.co/rest/v1/rpc/load_progress`
   - **Execution schedule:** *Every day*, at `06:00` (any time is fine).
4. **Advanced** tab:
   - **Request method:** `POST`
   - **Headers** (add two):

Key	Value
`apikey`	the team project's publishable key (from `src/config.ts`)
`Content-Type`	`application/json`

   - **Request body:** `{"p_code":"257F-T76R"}` (the jury's Mouth starting point)
5. Under **Notifications**, turn on **"Notify me when the job fails"**, so you hear about it if the project ever stops answering.
6. Click **Create** (or **Save**). Then open the job and use **Test run** (or wait for the first run).
7. Open the job's **History**: the run must show status **200 OK**. Its response is the jury save, a short JSON text that contains `"place": "mouth"` and `"template": true`.
8. Leave the job running until at least **30 November 2026** (the end of the judging, with a margin). Put a reminder in your calendar to decide then, with the team, whether to keep it.
9. **README:** add a section **"Keep-alive"** to `backend/README.md`, three or four lines: which service, which address, how often, that it only reads a jury save, who receives the failure emails, and until when it runs.

Screenshots to take. (1) the job's settings page (`keepalive-job.png`): crop or cover the apikey value if you prefer, even though it is the public key; (2) the history with a 200 OK run (`keepalive-history.png`).

5. Task 2: The new statistics

What is new in a save

The game now sends two more fields in `data`. Here is a save of a player who finished the whole game:

```
{
  "version": 1, "lang": "ar", "place": "complete",
  // NEW: the furthest place reached (replaying an early part does not move it back)
  "furthest": "complete",
  // NEW: each mission's BEST result: its stars (1 to 3) and its points
  //      ("revision" is the Memory Challenge: its score = memory moves, fewer is better)
  "results": {
    "mouth":           { "stars": 3, "score": 2100 },
    "stomach":         { "stars": 2, "score": 1100 },
    "small-intestine": { "stars": 3, "score": 1400 },
    "revision":        { "stars": 3, "score": 18 }
  },
  "predictions": { "saliva": "sugar", "pepsin": "protein", "bile": "bile" },
  "chapter3": { "lesson": 4, "quiz": 5, "score": 1400, "mistakes": 3 },
  "discoveries": [ "…" ], "hero": "nesma", "playerName": "", "savedAt": "…"
}
```

Field	Meaning
`furthest`	One of, in the order of the journey: `road`, `road2`, `mouth`, `esophagus`, `stomach`, `intestine-road`, `intestine-gate`, `small-intestine`, `revision`, `complete`. Older saves do not have it: use `place` for them.
`results`	Only the missions the player has finished. Missing = not finished yet.
`predictions`	The answer to the question a Guardian asks before each mission. The correct answers are `saliva` = `sugar`, `pepsin` = `protein`, `bile` = `bile`.
`chapter3.mistakes`	Wrong helpers used in the Small Intestine mission; `chapter3.lesson` = 4 means the mission is finished.
`template`	`true` only on the jury's starting points: always left out of statistics.

Steps

1. In **your own** project, open **SQL Editor**, paste the whole of `sample_saves.sql` and click **Run**. It adds 8 test saves (6 players, 2 jury starting points). Running it again resets them.
2. Open `backend/stats.sql` (your file) and paste the whole of `stats_round2.sql` **at its end**. Your four queries stay as they are; the new ones are numbered 5 to 8.
3. Run each new query on its own (select it, then **Run**; or paste one at a time). Compare with the expected results below: with the sample saves, they must be **exactly** these.
4. Read the comments in each query and make sure you understand them; the end of Section 8 explains the SQL pieces they use.

Expected results with the sample saves

5) Stars per mission

mission	players	average_stars	one_star	two_stars	three_stars
mouth	4	2.25	1	1	2
stomach	3	2.33	0	2	1
small-intestine	2	2.50	0	1	1
revision	1	3.00	0	0	1

6) How far players got (the six players, one at each place; the jury rows are left out)

furthest_place	players
road	1
esophagus	1
stomach	1
small-intestine	1
revision	1
complete	1

7) Predictions

question	correct_answer	answered	correct	percent_correct
bile	bile	3	2	66.7
pepsin	protein	4	2	50.0
saliva	sugar	5	3	60.0

8) Mistakes and memory moves

avg_intestine_mistakes	avg_memory_moves
6.0	18.0

Screenshots to take. The result of each query 5 to 8 in the SQL Editor: `stats-5.png`, `stats-6.png`, `stats-7.png`, `stats-8.png`.

For the README

Update the `stats.sql` line of the files table, and add a section **"Statistics (round 2)"**: one line per new query saying what it answers.

6. Task 3: Privacy note and clean-up

a) The privacy note: `backend/PRIVACY.md`

Write it for **pupils, parents and teachers**: short sentences, no technical words, 6 to 10 lines. Start from `PRIVACY_template.md` in the kit and check each sentence is true. It must say:

- **What is saved:** the chosen character, an optional first name, the language, and the progress in the game (places reached, discoveries, answers to the questions, stars).
- **What is never saved:** no email, no password, no account, no location, no photo; nothing that can identify a child unless they type their real name.
- **Where and why:** on the player's device, and online with Supabase (a database service), only so the player can continue on another device with their save code.
- **Who can see it:** only someone who has the code. The Funscientia team sees totals for the whole game (how many players, average stars), never used to identify anyone.
- **How long:** a save nobody has used for 12 months is deleted.
- **Tip:** "Use a nickname, not your full name."

Dounia and Hicham will translate it into Arabic and show it in the game.

b) The clean-up: `backend/cleanup.sql`

The file has three parts. **Test them on your own project only**, with the sample saves of Task 2 (run `sample_saves.sql` again first, so the test starts from the same 8 rows).

1. **Preview.** Run part 1. It must list **exactly one** save: `TEST-8888` (place `road`), untouched for 14 months. The old jury row `JURY-3456` must **not** be listed: templates are never deleted.
2. **Delete.** Run part 2. Supabase may warn that the query deletes rows: confirm. Then, in Table Editor, check that 7 rows are left, that `TEST-8888` is gone and that both `JURY-…` rows are still there.
3. **Schedule (optional, own project).** Enable the `pg_cron` extension (Database > Extensions; search "pg_cron"; Enable). Remove the `--` in front of the `cron.schedule` lines of part 3 and run them. Then run `select jobid, schedule, command from cron.job;`: you must see the job with the schedule `0 3 1 * *`. Then stop it with `select cron.unschedule('funscientia-cleanup');` (on your own project it is only a test).

> **Keep the `--` in front of part 3 in the file you deliver,** so that nobody schedules it by running the whole file without thinking. The team decides after the review.

Screenshots to take. The preview listing only `TEST-8888` (`cleanup-preview.png`); the Table Editor after the delete, with the 7 rows left (`cleanup-after.png`); and if you did step 3, the `cron.job` result (`cleanup-cron.png`).

For the README

Add a section **"Clean-up"**: what it deletes, what it never deletes, that it was tested on a test project, and that it is not scheduled on the team project yet. Add `cleanup.sql`, `PRIVACY.md` and `sample_saves.sql` to the files table.

7. Deliver your work

1. Copy the files into `backend/`: `stats.sql` (with the new queries), `cleanup.sql`, `PRIVACY.md`, `sample_saves.sql`, the updated `README.md`, and the screenshots into `backend/screenshots/`.
2. Check that you changed nothing outside `backend/`: `git status` must list only `backend/…` files.
3. Commit: `git add backend`, then `git commit -m "Backend round 2: keep-alive, new statistics, privacy note and clean-up"`.
4. Push: `git push -u origin backend-round2`.
5. On GitHub, open a **pull request** from `backend-round2` into `main`. In its description, paste the checklist below, ticked.
6. Send Hicham the link. **Do not merge it yourself**: we review it first, as last time.

Checklist for the pull request

- ☐ Keep-alive job on cron-job.org, daily, POST to `load_progress` with `257F-T76R`; history shows 200 OK; failure emails on; runs until at least 30 November
- ☐ Only the publishable key used; no secret or `service_role` key anywhere (files, screenshots, cron job)
- ☐ `stats.sql`: queries 5 to 8 added at the end; with the sample saves, the results match this guide exactly
- ☐ `cleanup.sql`: the preview lists only `TEST-8888`; after the delete, 7 rows are left and both jury rows remain; part 3 still commented out
- ☐ `cleanup.sql` never run on the team project
- ☐ `PRIVACY.md`: what is saved, what is never saved, where and why, who can see it, 12 months, the nickname tip
- ☐ `README.md`: sections Keep-alive, Statistics (round 2) and Clean-up; files table updated
- ☐ Screenshots in `backend/screenshots/`; only `backend/` changed

How we review, and what comes next

Hicham and Claude (our AI coding assistant) check every file and rerun your tests. Then we merge, Hicham runs your statistics on the team project for our submission, and the team decides on scheduling the clean-up. If something needs changing, we ask in the pull request.

8. Troubleshooting and SQL notes

What you see	What to do
cron-job.org history: **401** or **"Invalid API key"**	The `apikey` header is missing or mistyped. Copy the key again from `src/config.ts` (no spaces, no quotes). Header names are case-sensitive: `apikey`.
History: **404**	The address is wrong: it must end with `/rest/v1/rpc/load_progress`.
History: 200, but the response is `null`	The body is wrong. It must be exactly `{"p_code":"257F-T76R"}`, with the `Content-Type: application/json` header.
History: **timeout** or **5xx**	The team project may be paused. Tell Hicham at once: he restores it from the dashboard.
`sample_saves.sql`: "violates check constraint"	A code does not match the format: four characters, a dash, four characters, without 0, O, 1, I or L. The codes of the file are correct: paste it again, unchanged.
`sample_saves.sql`: "relation public.saves does not exist"	You are in a project without the table. Run `schema.sql`, `functions.sql` and `security.sql` first.
Your results differ from the expected ones	Run `sample_saves.sql` again (it resets the test rows), then the query. Check you pasted the whole query, from `select` (or `with`) to its `;`.
`cron.schedule`: "schema cron does not exist"	The `pg_cron` extension is not enabled yet (Database > Extensions).

The SQL pieces the new queries use

Piece	What it does
`data -> 'results'`	Takes a part of the JSON, still as JSON.
`data ->> 'place'`	Takes a part of the JSON as **text**.
`jsonb_each(…)`	Turns a JSON object into rows: one row per key (one per mission), with its value.
`jsonb_each_text(…)`	The same, with the values as text (used for the predictions).
`coalesce(a, b)`	`a`, or `b` when `a` is missing (older saves without `furthest` or `results`).
`count(*) filter (where …)`	Counts only the rows that meet the condition.
`array_position(array[…], x)`	The position of `x` in a list: used to sort places and missions in the order of the journey.
`nullif(n, 0)`	Avoids dividing by zero when nobody answered.
`now() - interval '12 months'`	The date one year ago: older saves are deleted.

Appendix: the full files

sample_saves.sql

```
-- sample_saves.sql: test saves for YOUR OWN Supabase project only (never the team project).
-- They look like real saves of today's game, including the new fields "results" and "furthest".
-- Run it in the SQL Editor; running it again resets them (it deletes the TEST and JURY rows first).

delete from public.saves where code like 'TEST-%' or code like 'JURY-%';

insert into public.saves (code, data, updated_at) values
  -- finished the whole game
  ('TEST-2222', '{"version":1,"lang":"ar","place":"complete","furthest":"complete",
     "results":{"mouth":{"stars":3,"score":2100},"stomach":{"stars":2,"score":1100},"small-
intestine":{"stars":3,"score":1400},"revision":{"stars":3,"score":18}},
     "predictions":{"saliva":"sugar","pepsin":"protein","bile":"bile"},
     "chapter3":{"lesson":4,"quiz":5,"score":1400,"mistakes":3}}', now()),
  -- in Chapter 3, before the Small Intestine mission
  ('TEST-3333', '{"version":1,"lang":"en","place":"small-intestine","furthest":"small-intestine",
     "results":{"mouth":{"stars":2,"score":1200},"stomach":{"stars":2,"score":900}},
     "predictions":{"saliva":"wet","pepsin":"all","bile":"lipase"},
     "chapter3":{"lesson":0,"quiz":0,"score":0,"mistakes":0}}', now()),
  -- in the Stomach mission
  ('TEST-4444', '{"version":1,"lang":"ar","place":"stomach","furthest":"stomach",
     "results":{"mouth":{"stars":1,"score":500}},
     "predictions":{"saliva":"sugar","pepsin":"starch"}}', now()),
  -- at the Memory Challenge
  ('TEST-5555', '{"version":1,"lang":"ar","place":"revision","furthest":"revision",
     "results":{"mouth":{"stars":3,"score":1800},"stomach":{"stars":3,"score":1300},"small-
intestine":{"stars":2,"score":1100}},
     "predictions":{"saliva":"nothing","pepsin":"protein","bile":"bile"},
     "chapter3":{"lesson":4,"quiz":0,"score":1100,"mistakes":9}}', now()),
  -- an older save, from before "results" and "furthest" existed
  ('TEST-6666', '{"version":1,"lang":"en","place":"esophagus","predictions":{"saliva":"sugar"}}',
now()),
  -- a jury starting point (a template): left out of every statistic
  ('JURY-2345', '{"version":1,"template":true,"place":"mouth","furthest":"mouth"}', now()),
  -- a save nobody has touched for 14 months: the clean-up must delete it
  ('TEST-8888', '{"version":1,"lang":"en","place":"road"}', now() - interval '14 months'),
  -- an old jury starting point: the clean-up must KEEP it (templates are never deleted)
  ('JURY-3456', '{"version":1,"template":true,"place":"stomach"}', now() - interval '14 months');
```

stats_round2.sql (to add at the end of `stats.sql`)

```
-- ===== Round 2 (October 2026): the new fields "results" and "furthest" =====
-- Read-only, like the queries above. Jury starting points (templates) are left out.

-- 5) Stars per mission: how many players finished it, their average stars, and how many got 1, 2 or
3.
--    (results.<mission>.stars is each player's BEST result; for "revision", score = memory moves)
select r.mission,
       count(*)                                              as players,
       round(avg((r.result ->> 'stars')::numeric), 2)        as average_stars,
       count(*) filter (where (r.result ->> 'stars') = '1') as one_star,
       count(*) filter (where (r.result ->> 'stars') = '2') as two_stars,
       count(*) filter (where (r.result ->> 'stars') = '3') as three_stars
```

```
from public.saves s,
     jsonb_each(coalesce(s.data -> 'results', '{}'::jsonb)) as r(mission, result)
where coalesce(s.data ->> 'template', 'false') <> 'true'
group by r.mission
order by array_position(array['mouth', 'stomach', 'small-intestine', 'revision'], r.mission);

-- 6) How far players got: the furthest place each one reached (older saves: where they are),
--    in the order of the journey.
select coalesce(s.data ->> 'furthest', s.data ->> 'place') as furthest_place,
       count(*)                                            as players
from public.saves s
where coalesce(s.data ->> 'template', 'false') <> 'true'
group by 1
order by array_position(array['road', 'road2', 'mouth', 'esophagus', 'stomach', 'intestine-road',
                              'intestine-gate', 'small-intestine', 'revision', 'complete'],
                        coalesce(s.data ->> 'furthest', s.data ->> 'place'));

-- 7) Predictions: for each question asked before a mission, how many players answered
--    and how many predicted correctly (the mission then shows them the answer).
with correct(question, answer) as (
  values ('saliva', 'sugar'), ('pepsin', 'protein'), ('bile', 'bile')
),
answers as (
  select p.question, p.answer
  from public.saves s,
       jsonb_each_text(coalesce(s.data -> 'predictions', '{}'::jsonb)) as p(question, answer)
  where coalesce(s.data ->> 'template', 'false') <> 'true'
)
select c.question,
       c.answer                                                          as correct_answer,
       count(a.answer)                                                   as answered,
       count(*) filter (where a.answer = c.answer)                       as correct,
       round(100.0 * count(*) filter (where a.answer = c.answer)
             / nullif(count(a.answer), 0), 1)                            as percent_correct
from correct c
left join answers a on a.question = c.question
group by c.question, c.answer
order by c.question;

-- 8) The Small Intestine mission and the Memory Challenge: average wrong helpers (mistakes) of the
--    players who finished the mission, and average memory moves of those who finished the challenge.
select round(avg((s.data -> 'chapter3' ->> 'mistakes')::numeric)
               filter (where (s.data -> 'chapter3' ->> 'lesson') = '4'), 1)
         as avg_intestine_mistakes,
       round(avg((s.data -> 'results' -> 'revision' ->> 'score')::numeric), 1)
         as avg_memory_moves
from public.saves s
where coalesce(s.data ->> 'template', 'false') <> 'true';
```

cleanup.sql

```
-- cleanup.sql: delete the saves nobody has used for 12 months (never the jury's starting points).
-- Run in the SQL Editor. Step 1 only LOOKS; step 2 deletes; step 3 (optional) does step 2 every
month.
-- On the team project: only after the team has reviewed this file.

-- 1) Preview: which saves would be deleted?
select code, updated_at, data ->> 'place' as place
```

```
from public.saves
where updated_at < now() - interval '12 months'
  and coalesce(data ->> 'template', 'false') <> 'true'
order by updated_at;

-- 2) Delete them (the same condition as the preview).
delete from public.saves
where updated_at < now() - interval '12 months'
  and coalesce(data ->> 'template', 'false') <> 'true';

-- 3) Optional: do step 2 automatically, at 03:00 on the 1st of every month.
--    First enable the "pg_cron" extension (Database > Extensions; search "pg_cron").
-- select cron.schedule(
--   'funscientia-cleanup',
--   '0 3 1 * *',
--   $$delete from public.saves
--     where updated_at < now() - interval '12 months'
--       and coalesce(data ->> 'template', 'false') <> 'true'$$
-- );
--    To check it:  select jobid, schedule, command from cron.job;
--    To stop it:   select cron.unschedule('funscientia-cleanup');
```

PRIVACY_template.md

```
# Funscientia: your privacy

Funscientia saves your game so you can continue it later, on this device or on another one.

**What we save:** the character you chose, the first name you typed (if you typed one), your
language,
and your progress in the game: the places you reached, your discoveries, your answers to the
Guardians'
questions and your stars.

**What we never save:** no email, no password, no account, no location, no photo.
Tip: use a nickname, not your full name.

**Where and why:** on your device, and online (with Supabase, a database service) only so that your
save code can bring your game back on another device.

**Who can see it:** only someone who has your save code. The Funscientia team looks only at totals
for
the whole game (for example, how many players finished it), never at who you are.

**How long:** a save that nobody has used for 12 months is deleted.
```